

Multiproceso en Java

Ejercicio de recapitulación

En esta actividad veremos el caso más habitual de uso del multiproceso en Java.

Queremos probar el generador de números pseudoaleatorios *Mersenne twister* implementado en la función `random()` de Python. La idea es generar un archivo muy grande de texto con caracteres aleatorios de la 'a' a la 'z' y contar las ocurrencias de cada carácter, viendo cual es el carácter más repetido y el que menos. Para un archivo suficientemente grande no deberían ser muy distintos.

Para generar el archivo emplearemos el script que se muestra a continuación:

```
#!/usr/bin/env python3

import sys
import random
import string

if len(sys.argv) != 3:
    print("Generador de archivos con letras aleatorias");
    print(f"Uso: {sys.argv[0]} <archivo> <bytes>")
    sys.exit(1)
nombre_archivo = sys.argv[1]
num_bytes = int(sys.argv[2])

caracteres = string.ascii_lowercase

with open(nombre_archivo, 'w') as archivo:
    for i in range(num_bytes):
        archivo.write(random.choice(caracteres))

print(f"Archivo '{nombre_archivo}' generado: {num_bytes} bytes")
```

Para el análisis, emplearemos utilidades del sistema operativo. Concretamente para Linux usaremos los comandos de shell `tr` y `wc`, de esta manera:

```
tr -dc 'carácter' < <archivo> | wc -m
```

Para Windows usaremos el siguiente comando de PowerShell:

```
Get-Content <archivo> | Select-String -Pattern "<caracter>" -
AllMatches | % Matches | Measure-Object | % Count
```

La idea es lanzar un proceso por cada una de las letras del abecedario ASCII, es decir, 26. Estos procesos se lanzarán en paralelo, sin tener en cuenta el número de procesadores, dejando que el sistema operativo los gestione mediante la cola de procesos.

El resultado que se mostrará es el número de veces que ha ocurrido cada letra, indicando cual es la letra que ha aparecido más veces y la que menos, con sus respectivos números de ocurrencias.

Este ejercicio se puede hacer en Windows o en Linux, a elegir.

Recomendaciones

- Para lanzar comandos en shell en Linux se puede usar:

```
ProcessBuilder("bash", "-c", "<comando>")
```

- Para lanzar comandos de PowerShell en Windows se puede usar:

```
ProcessBuilder("powershell.exe", "/command", "<comando>")
```

- La lectura de resultados no necesariamente tiene que hacerse según terminan los procesos (no tenemos la urgencia de detectar que un NEO va a destruir la Tierra), sino que se puede hacer cuando hayan terminado todos los procesos.
- En una primera versión no es necesario llamar al script de Python desde Java, se puede hacer únicamente usando el archivo generado previamente.